

Proyecto Final Integrador (Arquitectura Hexagonal/Limpia)

Objetivos:

- Construir un servicio de Orders con dominio, casos de uso puertos y adaptadores.
- Exponer API FastAPI segura y documentada.
- Asegurar calidad (pruebas, lint, tipado) y delivery (Docker, CI/CD).

Entregables:

- Código con capas (dominio, aplicación, infraestructura) y API.
- Pruebas unitarias, de contrato, integración/E2E; migraciones Alembic.
- Dockerfile multistage y pipeline CI; README y diagramas.
- Evidencia de auditoría de dependencias.

FASE 1: Inicialización del Proyecto, Estrategia Git y Entorno

Se comenzará creando el proyecto desde cero con Poetry, configurando la estructura de carpetas de Arquitectura Hexagonal y realizando nuestro primer commit.

Paso 1.1: Inicializar el proyecto con Poetry

- Ingresamos:
poetry new orders-service --src
cd orders-service

```
$ poetry new orders-service --src
The --src option is now the default and will be removed in a future version.
Created package orders_service in orders-service
```

Paso 1.2: Inicializar y configurar .gitignore

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course
$ git init
Initialized empty Git repository in C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/.git/
```

- Creamos nuestro archivo .gitignore:

```
.gitignore U X
orders-service > .gitignore
1  # Python
2  __pycache__/
3  *.py[cod]
4  *$py.class
5  .venv/
6  env/
7  build/
8  dist/
9  *.egg-info/
10
11 # Testing & Quality
12 .coverage
13 htmlcov/
14 .pytest_cache/
15 .mypy_cache/
16 .ruff_cache/
17
18 # Environment & Secrets
19 .env
20 .env.local
21
22 # IDEs
23 .vscode/
24 .idea/
```

Paso 1.3: Agregar las dependencias iniciales del proyecto:

Instalamos las dependencias esenciales agrupadas por producción y desarrollo:

- Dependencias de producción principales:

poetry add fastapi "uvicorn[standard]" pydantic-settings sqlalchemy alembic

```
$ poetry add fastapi "uvicorn[standard]" pydantic pydantic-settings sqlalchemy alembic
Using version ^0.142.2 for fastapi
Using version ^0.54.0 for uvicorn
Using version ^2.13.5 for pydantic
Using version ^2.15.0 for pydantic-settings
Using version ^2.1.1 for sqlalchemy
Using version ^1.20.0 for alembic

Updating dependencies
Resolving dependencies... (1.5s)

Package operations: 5 installs, 0 updates, 0 removals

- Installing markupsafe (3.0.3)
- Installing mako (1.4.3)
- Installing sqlalchemy (2.1.1)
- Installing alembic (1.20.0)
- Installing pydantic-settings (2.15.0)

Writing lock file
```

- Dependencias de desarrollo, calidad y testing:

Poetry add --group dev pytest pytest-cov mypy ruff httpx pip-audit

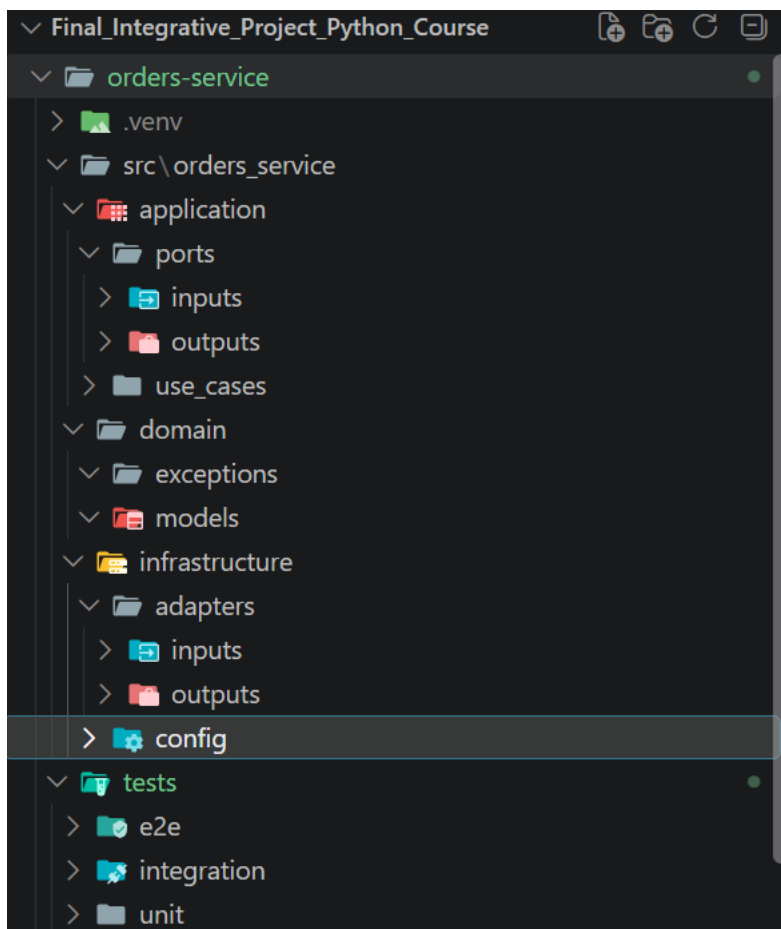
```
juan.rivera@mxjuriagdel15400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (master)
$ poetry add --group dev pytest pytest-cov mypy ruff httpx pip-audit
Using version ^9.1.1 for pytest
Using version ^7.1.0 for pytest-cov
Using version ^2.3.1 for mypy
Using version ^0.16.9 for ruff
Using version ^0.28.1 for httpx
Using version ^2.10.1 for pip-audit

Updating dependencies
Resolving dependencies... (1.7s)

Package operations: 40 installs, 0 updates, 0 removals

- Installing boolean-py (5.0)
- Installing certifi (2026.7.22)
- Installing charset-normalizer (3.5.2)
- Installing defusedxml (0.7.1)
- Installing mdurl (0.1.2)
- Installing urllib3 (2.8.0)
- Installing colorama (0.4.6)
- Installing filelock (4.0.7)
- Installing iniconfig (2.3.0)
- Installing markdown-it-py (4.2.0)
- Installing msgpack (1.2.3)
- Installing packageurl-python (0.17.6)
- Installing license-expression (30.4.4)
- Installing packaging (26.3)
- Installing pluggy (1.6.0)
```

Paso 1.4: Estructura de Capas (Arquitectura Hexagonal):



Paso 1.5: Crear el archivo de GitHub Actions para CI:

```

name: Continuous Integration

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

jobs:
  quality-and-tests:
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Set up Python 3.13
        uses: actions/setup-python@v5
        with:
          python-version: "3.13"

      - name: Install Poetry
        run: |
          curl -sSL https://install.python-poetry.org | python3 -
          echo "$HOME/.local/bin" >> $GITHUB_PATH

      - name: Install dependencies
        run: poetry install --no-interaction

      - name: Run Ruff (Linter)
        run: poetry run ruff check .

      - name: Run Mypy (Type Checking)
        run: poetry run mypy src

      - name: Run Pytest
        run: poetry run pytest

```

Paso 1.6: Vinculación con GitHub:

- Creamos el repositorio vacío en la cuenta de GitHub
- Conectamos el repositorio local y subimos los cambios.

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ git remote add origin https://github.com/juanrivera-A/proyecto-final-capacitacion-python.git
```

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ git push -u origin main
Enumerating objects: 4266, done.
Counting objects: 100% (4266/4266), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3664/3664), done.
Writing objects: 100% (4266/4266), 20.71 MiB | 4.71 MiB/s, done.
Total 4266 (delta 441), reused 4266 (delta 441), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (441/441), done.
To https://github.com/juanrivera-A/proyecto-final-capacitacion-python.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ git push -u origin main
Enumerating objects: 4266, done.
Counting objects: 100% (4266/4266), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3664/3664), done.
Writing objects: 100% (4266/4266), 20.71 MiB | 4.71 MiB/s, done.
Total 4266 (delta 441), reused 4266 (delta 441), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (441/441), done.
To https://github.com/juanrivera-A/proyecto-final-capacitacion-python.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Paso 1.7: Integrar pre-commit local

- Añadimos pre-commit a las dependencias dev para asegurar que nada roto pase localmente antes de hacer commit:

poetry add --group dev pre-commit

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry add --group dev pre-commit
Using version ^4.6.2 for pre-commit

Updating dependencies
Resolving dependencies... (0.5s)

Package operations: 7 installs, 0 updates, 38 removals

- Removing pip (26.2.1)
- Removing ast-serialize (0.11.2)
- Removing boolean-py (5.0)
- Removing cachecontrol (0.14.4)
- Removing certifi (2026.7.22)
- Removing charset-normalizer (3.5.2)
- Removing colorama (0.4.6)
- Removing coverage (7.16.2)
- Removing cyclonedx-python-lib (11.12.0)
- Removing defusedxml (0.7.1)
- Removing httpcore (1.0.9)
- Removing httpx (0.28.1)
- Removing iniconfig (2.3.0)
- Removing librt (0.16.0)
- Removing license-expression (30.4.4)
- Removing markdown-it-py (4.2.0)
- Removing mdurl (0.1.2)
- Removing msgpack (1.2.3)
- Removing mypy (2.3.1)
```

- Creamos el archivo **.pre-commit-config.yaml** en la raíz del proyecto.

```
.pre-commit-config.yaml U X
orders-service > .pre-commit-config.yaml
1  repos:
2    - repo: https://github.com/pre-commit/pre-commit-hooks
3      rev: v4.6.0
4    hooks:
5      - id: trailing-whitespace
6      - id: end-of-file-fixer
7      - id: check-yaml
8      - id: check-added-large-files
9
10   - repo: https://github.com/astral-sh/ruff-pre-commit
11     rev: v0.3.5
12     hooks:
13       - id: ruff
14         args: [--fix]
15       - id: ruff-format
```

- Instalamos los hooks en Git:

poetry run pre-commit install

```
$ poetry run pre-commit install
pre-commit installed at .git\hooks\pre-commit
```

- Subimos a GitHub y Validamos

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ git add .

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ git commit -m "test: añadir prueba de marcador de posición inicial para la canalización de CI"
trim trailing whitespace.....Passed
fix end of files.....Passed
check yaml.....(no files to check)Skipped
check for added large files.....Passed
ruff.....Passed
ruff-format.....Passed
[main 5bcb54e] test: añadir prueba de marcador de posición inicial para la canalización de CI
1 file changed, 2 insertions(+)
create mode 100644 orders-service/tests/test_placeholder.py
```

Continuous Integration

test: añadir prueba de marcador de posición inicial para la canalizac... #6 Re-run all jobs ...

Summary

All jobs

quality-and-tests

Run details

Usage

Workflow file

Triggered via push 3 minutes ago

juanrivera-A pushed 5bcb54e main

Status Success

Total duration 31s

Artifacts -

ci.yml

on: push

quality-and-tests 26s

Fase 2: Diseño del Dominio (TDD & Hexagonal Architecture)

En Arquitectura Hexagonal, el Dominio es el núcleo del microservicio. No conoce base de datos, no conoce FastAPI, no conoce marcos de trabajo web ni librerías externas. Es código Python puro enfocado exclusivamente en las reglas de negocio de los pedidos (Orders).

Objetivos de la “Fase 2”:

1. Definir los Modelos de Dominio / Entidades (Order, OrderItem, OrderStatus)
2. Definir las Excepciones de Dominio (ej. InvalidOrderStateError, EmptyOrderError).
3. Definir los Value Objects si aplicasen (ej. Money, Quantity).
4. Implementar todo mediante TDD (Red-Green-Refactor)

Paso 1: Primer Test de Dominio (Red Phase)

Creemos nuestro primer archivo de prueba real de negocio (orders-service/tests/unit/domain/test_order.py)

```
from decimal import Decimal

import pytest
from src.orders_service.domain.exceptions import EmptyOrderError
from src.orders_service.domain.models import Order, OrderItem, OrderStatus

def test_create_order_successfully() -> None:
    items = [
        OrderItem(product_id="prod-1", quantity=2,
price=Decimal("100.00")),
        OrderItem(product_id="prod-2", quantity=1,
price=Decimal("50.00")),
    ]

    order = Order(customer_id="cust-123", items=items)

    assert order.customer_id == "cust-123"
    assert order.status == OrderStatus.PENDING
    assert order.total_amount == Decimal("250.00")
    assert len(order.items) == 2

def test_create_order_without_items_raises_error() -> None:
    with pytest.raises(EmptyOrderError):
```

```
Order(customer_id="cust-123", items=[])
```

- Si ejecutamos poetry run pytest en la terminal, la prueba va a fallar (Red Phase) porque los módulos orders_service.domain.models y orders_service.domain.exceptions aún no existen.

```
$ poetry run pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service
configfile: pyproject.toml
plugins: anyio-4.15.1, platformdirs-4.12.2
collected 1 item / 1 error

===== ERRORS =====
_____ ERROR collecting tests/unit/domain/test_order.py _____
ImportError while importing test module 'C:\Users\Juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service\tests\unit\domain\test_order.py':
'.'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
tests\unit\domain\test_order.py:4: in <module>
    from src.orders_service.domain.exceptions import EmptyOrderError
E ImportError: cannot import name 'EmptyOrderError' from 'src.orders_service.domain.exceptions' (unknown location)
===== short test summary info =====
ERROR tests/unit/domain/test_order.py
!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!
===== 1 error in 0.23s =====
```

Paso 2: Implementación del Dominio (Green Phase)

Escribimos el código de producción necesario dentro del paquete (src/orders_service/domain/).

- Excepciones (src/orders_service/domain/exceptions.py)

```
test_order.py 2, U  exceptions.py U X
orders-service > src > orders_service > domain > exceptions.py > EmptyOrderError > __init__
1 class DomainError(Exception):
2     """Base exception for domain layer errors."""
3
4
5 class EmptyOrderError(DomainError):
6     """Raised when an order is created without items."""
7
8     def __init__(self, message: str = "An order must contain at least one item.") -> None:
9         super().__init__(message)
```

- Modelos de Dominio (src/orders_service/domain/models.py)

Uso de dataclasses nativas de Python para mantener el dominio completamente desacoplado de orm/frameworks:

```
import uuid
from dataclasses import dataclass, field
from decimal import Decimal
from enum import Enum

from orders_service.domain.exceptions import EmptyOrderError

class OrderStatus(str, Enum):
    PENDING = "PENDING"
    PAID = "PAID"
    CANCELLED = "CANCELLED"

@dataclass(frozen=True)
class OrderItem:
    product_id: str
    quantity: int
    price: Decimal

    @property
    def subtotal(self) -> Decimal:
        return self.price * self.quantity

@dataclass
class Order:
    customer_id: str
```

```

items: list[OrderItem]
id: str = field(default_factory=lambda: str(uuid.uuid4()))
status: OrderStatus = OrderStatus.PENDING

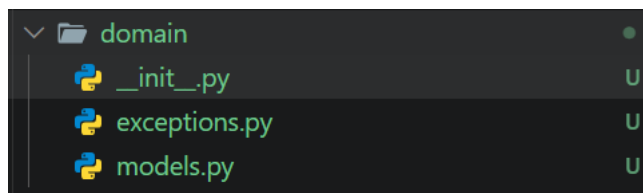
def __post_init__(self) -> None:
    if not self.items:
        raise EmptyOrderError()

@property
def total_amount(self) -> Decimal:
    return sum((item.subtotal for item in self.items),
Decimal("0.00"))

```

- Archivo `__init__.py` (src/orders_service/domain/ `__init__.py`)

Nos aseguramos de tener un `__init__.py` vacío.



Paso 3: Ejecución local de calidad:

Ejecutamos los comandos de validación en la terminal.

```

juan.rivera@mxjuriagdel15400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
• $ poetry run ruff check .
All checks passed!

juan.rivera@mxjuriagdel15400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
• $ poetry run mypy src
Success: no issues found in 4 source files

juan.rivera@mxjuriagdel15400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
• $ poetry run pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service
configfile: pyproject.toml
plugins: anyio-4.15.1, platformdirs-4.12.2
collected 3 items

tests\test_placeholder.py . [ 33%]
tests\unit\domain\test_order.py .. [100%]

===== 3 passed in 0.06s =====

```

Fase 2: Reglas de Negocio (Transiciones de Estado)

En un dominio robusto, el estado de un pedido no se debería de modificar cambiando el atributo directamente (como `order.status = "PAID"`). Debe hacerse a través de métodos explícitos de dominio que validen las reglas de negocio.

Por ejemplo:

- Un pedido solo puede pagarse si está en estado PENDING. Si ya está pagado o cancelado, debe lanzar una excepción (`InvalidOrderStateError`).
- Un pedido no se puede cancelar si ya fue pagado.

Paso 1: Escribir los tests para el comportamiento de negocio (Red Phase)

Añadimos las pruebas al final de nuestro archivo (`tests/unit/domain/tests_order.py`)

```

def test_pay_order_successfully() -> None:
    items = [OrderItem(product_id="prod-1", quantity=1,
price=Decimal("100.00"))]
    order = Order(customer_id="cust-123", items=items)

    order.pay()

    assert order.status == OrderStatus.PAID

def test_pay_already_paid_order_raises_error() -> None:
    items = [OrderItem(product_id="prod-1", quantity=1,
price=Decimal("100.00"))]
    order = Order(customer_id="cust-123", items=items)
    order.pay()

```

```

    with pytest.raises(InvalidOrderStateError):
        order.pay()

def test_cancel_order_successfully() -> None:
    items = [OrderItem(product_id="prod-1", quantity=1,
price=Decimal("100.00"))]
    order = Order(customer_id="cust-123", items=items)

    order.cancel()

    assert order.status == OrderStatus.CANCELLED

def test_cancel_paid_order_raises_error() -> None:
    items = [OrderItem(product_id="prod-1", quantity=1,
price=Decimal("100.00"))]
    order = Order(customer_id="cust-123", items=items)
    order.pay()

    with pytest.raises(InvalidOrderStateError):
        order.cancel()

```

- Y cuando ejecutamos poetry run pytest, fallara por que el método .pay(), .cancel() y la excepción aún no existen.

```

juan.rivera@mxjuriagdel15400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service
configfile: pyproject.toml
plugins: anyio-4.15.1, platformdirs-4.12.2
collected 1 item / 1 error

===== ERRORS =====
_____ ERROR collecting tests/unit/domain/test_order.py _____
ImportError while importing test module 'C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service\tests\unit\domain\test_order.py':
cannot import name 'InvalidOrderStateError' from 'orders_service.domain.exceptions' (C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service/src/orders_service/domain/exceptions.py)
Hint: make sure your test modules/packages have valid Python names.
Traceback:
...\.AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
...
tests/unit/domain/test_order.py:4: in <module>
    from orders_service.domain.exceptions import EmptyOrderError, InvalidOrderStateError
E ImportError: cannot import name 'InvalidOrderStateError' from 'orders_service.domain.exceptions' (C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service/src/orders_service/domain/exceptions.py)
===== short test summary info =====
ERROR tests/unit/domain/test_order.py
!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!
===== 1 error in 0.21s =====

```

Paso 2. Implemetar los métodos en el Dominio (Green Phase)

- En src/orders_service/domain/exceptions.py agregamos:

```
class InvalidOrderStateError(DomainError):
    """Raised when an operation is invalid for the current order state."""

    def __init__(self, message: str = "Invalid action for current order status.") -> None:
        super().__init__(message)
```

- En src/orders_service/domain/models.py agregamos los métodos a la clase Order:

```
def pay(self) -> None:
    if self.status != OrderStatus.PENDING:
        raise InvalidOrderStateError(f"Cannot pay an order with status {self.status.value}")
    self.status = OrderStatus.PAID

def cancel(self) -> None:
    if self.status == OrderStatus.PAID:
        raise InvalidOrderStateError("Cannot cancel an already paid order")
    self.status = OrderStatus.CANCELLED
```

Paso 3: Ejecución local de calidad.

Ejecutamos los comandos de validación en la terminal.

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run ruff check .
All checks passed!

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run mypy src
Success: no issues found in 4 source files

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service
configfile: pyproject.toml
plugins: anyio-4.15.1, platformdirs-4.12.2
collected 7 items

tests\test_placeholder.py . [ 14%]
tests\unit\domain\test_order.py ..... [100%]

===== 7 passed in 0.07s =====
```

Fase 2: Definición de Puertos del Dominio (Ports)

En Arquitectura Hexagonal, el Dominio define contratos e interfaces (Puertos) para la persistencia o servicios externos, pero no le importa cómo se implementan.

Por ejemplo, el puerto `OrderRepository` define las operaciones básicas que la capa de persistencia debe cumplir (guardar, buscar, actualizar), independientemente de si usamos PostgreSQL, SQLite o una base de datos en memoria.

Paso 1: Escribir la interfaz abstracta del repositorio

Creemos el archivo `src/orders_service/domain/ports.py` y definimos la clase base abstracta (Abstract Base Class –ABC)

```
ports.py U X
orders-service > src > orders_service > domain > ports.py > ...
1  from abc import ABC, abstractmethod
2
3  from orders_service.domain.models import Order
4
5
6  class OrderRepository(ABC):
7      """Abstract port for order persistence operations."""
8
9      @abstractmethod
10     def save(self, order: Order) -> None:
11         """Persists a new order or updates an existing one."""
12
13     @abstractmethod
14     def get_by_id(self, order_id: str) -> Order | None:
15         """Retrieves an order by its unique identifier."""
16
17     @abstractmethod
18     def get_all(self) -> list[Order]:
19         """Retrieves all orders."""
```

Paso 2. Test del contrato del puerto (opcional con Mock Repository)

Para asegurarnos de que la interfaz está bien estructurada y de que Mypy valida las abstracciones sin problemas, crearemos una prueba ligera usando un repositorio en memoria dentro de las pruebas del dominio.

```
from decimal import Decimal

from orders_service.domain.models import Order, OrderItem
from orders_service.domain.ports import OrderRepository

class InMemoryOrderRepository(OrderRepository):
```

```

    """In-memory implementation of OrderRepository for testing domain
    ports."""

    def __init__(self) -> None:
        self._orders: dict[str, Order] = {}

    def save(self, order: Order) -> None:
        self._orders[order.id] = order

    def get_by_id(self, order_id: str) -> Order | None:
        return self._orders.get(order_id)

    def get_all(self) -> list[Order]:
        return list(self._orders.values())

def test_in_memory_repository_save_and_get() -> None:
    repo = InMemoryOrderRepository()
    items = [OrderItem(product_id="prod-1", quantity=1,
price=Decimal("100.00"))]
    order = Order(customer_id="cust-123", items=items)

    repo.save(order)
    retrieved = repo.get_by_id(order.id)

    assert retrieved is not None
    assert retrieved.id == order.id
    assert len(repo.get_all()) == 1

```

Paso 3: Ejecución local de calidad.

Ejecutamos los comandos de validación en la terminal.

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run ruff check . --fix

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run mypy src
Success: no issues found in 5 source files

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service
configfile: pyproject.toml
plugins: anyio-4.15.1, platformdirs-4.12.2
collected 8 items

tests\test_placeholder.py . [ 12%]
tests\unit\domain\test_order.py ..... [ 87%]
tests\unit\domain\test_ports.py . [100%]

===== 8 passed in 0.09s =====
```

Fase 3: Casos de Uso & Aplicación

Siguiendo el flujo del proyecto y la Arquitectura Hexagonal, la capa de Aplicación orquesta la lógica de negocio consumiendo los Puertos del Dominio.

Objetivo de la “Fase 3”

- Definir CreateOrderUseCase (Creación de pedidos)
- Definir PayOrderUseCase (Pago de pedidos)
- Definir CancelOrderUseCase (Cancelación de pedidos)
- Definir GetOrderUseCase / ListOrderUseCase (Consulta de pedidos)

También se definirán los DTOs (Data Transfer Objects) de aplicación para la entrada y salida de datos sin acoplarnos a los modelos de transporte de la API.

Paso 1 de la Fase 3: DTOs de Aplicación (Red-Green-Refactor).

Se comenzará definiendo las estructuras de datos que recibirán y retornarán nuestros Casos de Uso (src/orders_service/application/dtos.py).

```

dtos.py U X
orders-service > src > orders_service > application > dtos.py > OrderResponseDTO
1  from dataclasses import dataclass
2  from decimal import Decimal
3
4
5  @dataclass(frozen=True)
6  class OrderItemDTO:
7      product_id: str
8      quantity: int
9      price: Decimal
10
11
12  @dataclass(frozen=True)
13  class CreateOrderInputDTO:
14      customer_id: str
15      items: list[OrderItemDTO]
16
17
18  @dataclass(frozen=True)
19  class OrderResponseDTO:
20      id: str
21      customer_id: str
22      status: str
23      total_amount: Decimal
24      items: list[OrderItemDTO]

```

Paso 2: Crear los Casos de Uso.

(src/orders_service/application/use_cases.py)

```

from orders_service.application.dtos import (
    CreateOrderInputDTO,
    OrderItemDTO,
    OrderResponseDTO,
)
from orders_service.domain.exceptions import DomainError
from orders_service.domain.models import Order, OrderItem
from orders_service.domain.ports import OrderRepository

class OrderUseCases:
    """Application service orchestration for order operations."""

    def __init__(self, repository: OrderRepository) -> None:
        self._repository = repository

```

```

    def create_order(self, input_dto: CreateOrderInputDTO) ->
OrderResponseDTO:
        domain_items = [
            OrderItem(product_id=item.product_id,
quantity=item.quantity, price=item.price)
            for item in input_dto.items
        ]
        order = Order(customer_id=input_dto.customer_id,
items=domain_items)
        self._repository.save(order)
        return self._to_response_dto(order)

    def pay_order(self, order_id: str) -> OrderResponseDTO:
        order = self._get_order_or_raise(order_id)
        order.pay()
        self._repository.save(order)
        return self._to_response_dto(order)

    def cancel_order(self, order_id: str) -> OrderResponseDTO:
        order = self._get_order_or_raise(order_id)
        order.cancel()
        self._repository.save(order)
        return self._to_response_dto(order)

    def get_order_by_id(self, order_id: str) -> OrderResponseDTO |
None:
        order = self._repository.get_by_id(order_id)
        if not order:
            return None
        return self._to_response_dto(order)

    def list_orders(self) -> list[OrderResponseDTO]:
        orders = self._repository.get_all()
        return [self._to_response_dto(order) for order in orders]

    def _get_order_or_raise(self, order_id: str) -> Order:
        order = self._repository.get_by_id(order_id)
        if not order:
            raise DomainError(f"Order with id {order_id} not found.")
        return order

```

```

def _to_response_dto(self, order: Order) -> OrderResponseDTO:
    return OrderResponseDTO(
        id=order.id,
        customer_id=order.customer_id,
        status=order.status.value,
        total_amount=order.total_amount,
        items=[
            OrderItemDTO(product_id=item.product_id,
quantity=item.quantity, price=item.price)
            for item in order.items
        ],
    )

```

Paso 3: Pruebas Unitarias de Casos de Uso

(tests/unit/application/test_use_cases.py)

```

from decimal import Decimal

import pytest
from orders_service.application.dtos import CreateOrderInputDTO,
OrderItemDTO
from orders_service.application.use_cases import OrderUseCases
from orders_service.domain.exceptions import DomainError
from tests.unit.domain.test_ports import InMemoryOrderRepository

@pytest.fixture
def use_cases() -> OrderUseCases:
    repo = InMemoryOrderRepository()
    return OrderUseCases(repository=repo)

def test_create_order_use_case(use_cases: OrderUseCases) -> None:
    input_dto = CreateOrderInputDTO(
        customer_id="cust-001",
        items=[OrderItemDTO(product_id="prod-100", quantity=2,
price=Decimal("150.00"))],
    )

    response = use_cases.create_order(input_dto)

```

```

    assert response.customer_id == "cust-001"
    assert response.status == "PENDING"
    assert response.total_amount == Decimal("300.00")

def test_pay_order_use_case(use_cases: OrderUseCases) -> None:
    input_dto = CreateOrderInputDTO(
        customer_id="cust-001",
        items=[OrderItemDTO(product_id="prod-100", quantity=1,
price=Decimal("50.00"))],
    )
    created = use_cases.create_order(input_dto)

    paid_response = use_cases.pay_order(created.id)

    assert paid_response.status == "PAID"

def test_pay_non_existent_order_raises_error(use_cases: OrderUseCases)
-> None:
    with pytest.raises(DomainError):
        use_cases.pay_order("non-existent-id")

```

Paso 3: Ejecución local de calidad.

Ejecutamos los comandos de validación en la terminal.

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run ruff check . --fix

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run ruff check .
All checks passed!

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run mypy src
Success: no issues found in 7 source files

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service
configfile: pyproject.toml
plugins: anyio-4.15.1, platformdirs-4.12.2
collected 10 items

tests\unit\application\test_use_cases.py ... [ 30%]
tests\unit\domain\test_order.py ..... [ 90%]
tests\unit\domain\test_ports.py . [100%]

===== 10 passed in 0.08s =====

```

Fase 4: Persistencia & Adaptadores Secundarios (SQLAlchemy 2.0 & Alembic)

En esta fase se conectara la aplicación con un base datos real (utilizando SQLite para desarrollo/testing e interfaz lista para PostgreSQL/MySQL en producción) mediante SQLAlchemy 2.0 (modelos ORM y sesiones) y Alembic (migraciones).

Objetivos de la “Fase 4”:

- Implementación del adaptador concreto de persistencia:
 - Crear la clase SQLAlchemyOrderRepository implementando la interfaz abstracta OrderRepository (Puerto del Dominio).
 - Mapear bidireccionalmete los modelos ORM (OrderModel, OrderItemModel) a las entidades puras del dominio (Order, OrderItem).
- Gestión de Esquemas y Migraciones con Alembic:
 - Inicializar Alembic y configurar el entorno de migraciones acomplado a la base Declarativa (Base.metadate).
 - Generar y aplicar la primera migración automatica (alembic revision – autogenerate / alembic upgrade head).
- Garantía de Calidad e Integración (Pruebas de Integración):
 - Configurar fixtures en Pytest para manejar sesiones de base de datos aisladas y transacciones durante las pruebas.
 - Escribir pruebas de integración (tests/integration/infrastructure/test_respository.py) para validad operaciones reales de lectura/escritura en base de datos.

Paso 1: Verificamos las dependencias a Poetry

poetry add sqlalchemy alembic

```
• $ poetry add sqlalchemy alembic
The following packages are already present in the pyproject.toml and will be skipped:

- sqlalchemy
- alembic

If you want to update it to the latest compatible version, you can use `poetry update package`.
If you prefer to upgrade it to the latest available version, you can use `poetry add package@latest`.

Nothing to add.
```

Paso 2: Definir la configuración de la Base de Datos

Creamos el archivo src/order_service/infrastructure/database.py

```

orders-service > src > orders_service > infrastructure > database.py > ...
1  from collections.abc import Generator
2
3  from sqlalchemy import create_engine
4  from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker
5
6  DATABASE_URL = "sqlite:///./orders.db"
7
8  engine = create_engine(
9      DATABASE_URL,
10     connect_args={"check_same_thread": False}, # Requerido solo para SQLite
11 )
12
13 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
14
15
16 class Base(DeclarativeBase):
17     """Base declarative class for SQLAlchemy ORM models."""
18
19
20
21 def get_db_session() -> Generator[Session, None, None]:
22     """Yields a database session and ensures proper cleanup."""
23     db = SessionLocal()
24     try:
25         yield db
26     finally:
27         db.close()

```

Paso 3: Definir los Modelos ORM (Mapeo de Base de Datos)

Creemos el archivo src/orders_service/infrastructure/models.py

```

from decimal import Decimal

from orders_service.infrastructure.database import Base
from sqlalchemy import ForeignKey, Numeric, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

class OrderModel(Base):
    __tablename__ = "orders"

    id: Mapped[str] = mapped_column(String(36), primary_key=True)

```

```

        customer_id: Mapped[str] = mapped_column(String(100),
nullable=False)
        status: Mapped[str] = mapped_column(String(20), nullable=False)

        items: Mapped[list["OrderItemModel"]] = relationship(
            "OrderItemModel",
            back_populates="order",
            cascade="all, delete-orphan",
        )

class OrderItemModel(Base):
    __tablename__ = "order_items"

    id: Mapped[int] = mapped_column(primary_key=True,
autoincrement=True)
    order_id: Mapped[str] = mapped_column(ForeignKey("orders.id"),
nullable=False)
    product_id: Mapped[str] = mapped_column(String(100),
nullable=False)
    quantity: Mapped[int] = mapped_column(nullable=False)
    price: Mapped[Decimal] = mapped_column(Numeric(10, 2),
nullable=False)

    order: Mapped[OrderModel] = relationship("OrderModel",
back_populates="items")

```

Paso 4: Verificación inicial

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/ju
● $ poetry run ruff check . --fix
Found 2 errors (2 fixed, 0 remaining).

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/ju
● $ poetry run mypy src
Success: no issues found in 9 source files

```

Fase 2: Implementación del Adaptador Concreto (SQLAlchemyOrderRepository)

El siguiente paso es crear el adaptador secundario que implementa la interfaz OrderRepository. Este repositorio se encargará de traducir entre las entidades puras del

dominio (Order, OrderItem) y los modelos ORM de la base de datos (OrderModel, OrderItemModel).

Paso 1: Creamos el archivo src/orders_service/infrastructure/repositories.py

```
from orders_service.domain.models import Order, OrderItem, OrderStatus
from orders_service.domain.ports import OrderRepository
from orders_service.infrastructure.models import OrderItemModel,
OrderModel
from sqlalchemy.orm import Session

class SQLAlchemyOrderRepository(OrderRepository):
    """SQLAlchemy implementation of the OrderRepository port."""

    def __init__(self, session: Session) -> None:
        self._session = session

    def save(self, order: Order) -> None:
        existing_model =
self._session.query(OrderModel).filter_by(id=order.id).first()

        if existing_model:
            existing_model.status = order.status.value
            existing_model.items.clear()
            existing_model.items = [
                OrderItemModel(
                    product_id=item.product_id,
                    quantity=item.quantity,
                    price=item.price,
                )
                for item in order.items
            ]
        else:
            new_model = OrderModel(
                id=order.id,
                customer_id=order.customer_id,
                status=order.status.value,
                items=[
                    OrderItemModel(
                        product_id=item.product_id,
                        quantity=item.quantity,
                        price=item.price,
```

```

        )
        for item in order.items
    ],
)
self._session.add(new_model)

self._session.commit()

def get_by_id(self, order_id: str) -> Order | None:
    model =
self._session.query(OrderModel).filter_by(id=order_id).first()
    if not model:
        return None
    return self._to_domain(model)

def get_all(self) -> list[Order]:
    models = self._session.query(OrderModel).all()
    return [self._to_domain(model) for model in models]

def _to_domain(self, model: OrderModel) -> Order:
    items = [
        OrderItem(
            product_id=item.product_id,
            quantity=item.quantity,
            price=item.price,
        )
        for item in model.items
    ]
    return Order(
        id=model.id,
        customer_id=model.customer_id,
        items=items,
        status=OrderStatus(model.status),
    )

```

Paso 2: Configuración de Alembic para Migraciones

Inicializaremos la herramienta de migraciones para que rastree los cambios de modelo en la base de datos.

- Inicializamos Alembic

```
$ poetry run alembic init alembic
Creating directory C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service\alembic ... done
Creating directory C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service\alembic\versions ... done
Generating C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service\alembic.ini ... done
Generating C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service\alembic\env.py ... done
Generating C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service\alembic\README ... done
Generating C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service\alembic\script.py.mako ... done
Please edit configuration/connection/logging settings in C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service\alembic.ini before proceeding.
```

- Configuramos alembic/env.py

Abrimos el archivo alembic/env.py e integramos la base de datos declarativa para habilitar la autogeneración de migraciones

```
24 target_metadata = Base.metadata
```

```
✓ from logging.config import fileConfig

from alembic import context
from orders_service.infrastructure.models import ( # noqa: F401
    Base,
    OrderItemModel,
    OrderModel,
)
from sqlalchemy import engine_from_config, pool
```

- Generamos la primera migración

```
$ poetry run alembic revision --autogenerate -m "crear las tablas orders y order_items"
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.schemas
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.tables
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.types
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.constraints
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.defaults
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.comments
INFO [alembic.autogenerate.compare.tables] Detected added table 'orders'
INFO [alembic.autogenerate.compare.tables] Detected added table 'order_items'
Generating C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service\alembic\versions\f9be9fe2ab59_crear_las_tablas_orders_y_order_items.py ... done
```

- Aplicamos la migración sobre la base de datos:

```
$ poetry run alembic upgrade head
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running upgrade -> f9be9fe2ab59, crear las tablas orders y order_items
```

Paso 4: Pruebas de Integración para el Repositorio Concreto.

Creamos la carpeta de pruebas de integración y el archivo correspondiente.

tests/integration/infrastructure/test_repository.py

```
from decimal import Decimal

import pytest
from orders_service.domain.models import Order, OrderItem, OrderStatus
from orders_service.infrastructure.database import Base
from orders_service.infrastructure.repositories import
SQLAlchemyOrderRepository
from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker

@pytest.fixture
def db_session() -> Session:
    engine = create_engine("sqlite:///memory:")
    Base.metadata.create_all(engine)
    SessionLocal = sessionmaker(bind=engine)
    session = SessionLocal()
    try:
        yield session
    finally:
        session.close()

def test_sqlalchemy_repository_save_and_get(db_session: Session) ->
None:
    repo = SQLAlchemyOrderRepository(session=db_session)
    items = [OrderItem(product_id="prod-abc", quantity=2,
price=Decimal("250.00"))]
    order = Order(customer_id="cust-xyz", items=items)

    repo.save(order)
    retrieved = repo.get_by_id(order.id)

    assert retrieved is not None
    assert retrieved.id == order.id
    assert retrieved.customer_id == "cust-xyz"
    assert retrieved.status == OrderStatus.PENDING
    assert retrieved.total_amount == Decimal("500.00")
```

```
def test_sqlalchemy_repository_update_status(db_session: Session) ->
None:
    repo = SQLAlchemyOrderRepository(session=db_session)
    items = [OrderItem(product_id="prod-1", quantity=1,
price=Decimal("100.00"))]
    order = Order(customer_id="cust-1", items=items)

    repo.save(order)
    order.pay()
    repo.save(order)

    updated = repo.get_by_id(order.id)
    assert updated is not None
    assert updated.status == OrderStatus.PAID
```

Paso 5: Verificación local.

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run ruff check .
All checks passed!

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run mypy src
Success: no issues found in 10 source files

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service
configfile: pyproject.toml
plugins: anyio-4.15.1, platformdirs-4.12.2
collected 12 items

tests\integration\infrastructure\test_repository.py .. [ 16%]
tests\unit\application\test_use_cases.py ... [ 41%]
tests\unit\domain\test_order.py ..... [ 91%]
tests\unit\domain\test_ports.py . [100%]

===== 12 passed in 0.58s =====
```

Fase 5: Adaptadores Primarios / API REST con FastAPI

Aquí expondremos la lógica de negocio a través de endpoints HTTP RESTful, asegurando que la API esté documentada, validada y acoplada a la capa de aplicación mediante Inyección de Dependencias.

Objetivos de la “Fase 5”:

- Esquemas de Transporte (Pydantic / OpenAPI)
 - Crear esquemas de solicitud (Request) y respuesta (Response) con validación de dato estricta.
- Controladores / Endpoints REST:
 - Crear el router de pedidos con rutas para: POST / orders, GET /orders/{id}, POST /orders/{id}/pay, POST /orders/{id}/ cancel y GET / orders.
 - Mapear excepciones de dominio a respuestas HTTP estandarizadas (404 Not Found, 400 Bad Request, 500 Internal Server Error)
- Inyección de Dependencias (fastapi.Depends):
 - Configurar el proveedor de sesiones de base de datos y la instancia de OrderUseCases por petición.
- Pruebas de integración de API (TestClient / pytest):
 - Validar el ciclo completo HTTP -> Casos de uso -> Base de datos.

Paso 1: Verificamos las dependencias FastAPI y Uvicorn.

poetry add fastapi uvicorn

```
$ poetry add fastapi uvicorn
The following packages are already present in the pyproject.toml and will be skipped:

- fastapi
- uvicorn

If you want to update it to the latest compatible version, you can use `poetry update package`.
If you prefer to upgrade it to the latest available version, you can use `poetry add package@latest`.

Nothing to add.
```

Paso 2: Definir los Esquemas Pydantic para la API.

Creamos el archivo src/orders_service/infrastructure/api/schemas.py.

```
from decimal import Decimal

from pydantic import BaseModel, Field
```

```

class OrderItemRequest(BaseModel):
    product_id: str = Field(..., min_length=1, description="ID único del producto")
    quantity: int = Field(..., gt=0, description="Cantidad del producto (mayor a 0)")
    price: Decimal = Field(..., gt=0, description="Precio unitario (mayor a 0)")

class CreateOrderRequest(BaseModel):
    customer_id: str = Field(..., min_length=1, description="ID del cliente")
    items: list[OrderItemRequest] = Field(..., min_items=1, description="Lista de productos")

class OrderItemResponse(BaseModel):
    product_id: str
    quantity: int
    price: Decimal

class OrderResponse(BaseModel):
    id: str
    customer_id: str
    status: str
    total_amount: Decimal
    items: list[OrderItemResponse]

```

Paso 3: Configurar la inyección de dependencias.

Creemos el archivo `src/orders_service/infrastructure/api/dependencies.py`

```

from fastapi import Depends
from orders_service.application.use_cases import OrderUseCases
from orders_service.infrastructure.database import get_db_session
from orders_service.infrastructure.repositories import SQLAlchemyOrderRepository
from sqlalchemy.orm import Session

```

```
def get_use_cases(db: Session = Depends(get_db_session)) ->
OrderUseCases:
    repository = SQLAlchemyOrderRepository(session=db)
    return OrderUseCases(repository=repository)
```

Paso 4: Crear el Router de Pedidos

Creamos el archivo src/orders_service/infrastructure/api/routers.py

```
from fastapi import APIRouter, Depends, HTTPException, status
from orders_service.application.dtos import CreateOrderInputDTO,
OrderItemDTO
from orders_service.application.use_cases import OrderUseCases
from orders_service.domain.exceptions import DomainError
from orders_service.infrastructure.api.dependencies import
get_use_cases
from orders_service.infrastructure.api.schemas import
CreateOrderRequest, OrderResponse

router = APIRouter(prefix="/orders", tags=["Orders"])

@router.post("", response_model=OrderResponse,
status_code=status.HTTP_201_CREATED)
def create_order(
    payload: CreateOrderRequest,
    use_cases: OrderUseCases = Depends(get_use_cases),
) -> OrderResponse:
    try:
        input_dto = CreateOrderInputDTO(
            customer_id=payload.customer_id,
            items=[
                OrderItemDTO(product_id=item.product_id,
quantity=item.quantity, price=item.price)
                for item in payload.items
            ],
        )
        return use_cases.create_order(input_dto) # type:
ignore[return-value]
    except DomainError as err:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail=str(err)) from err
```

```

@router.get("/{order_id}", response_model=OrderResponse)
def get_order(
    order_id: str,
    use_cases: OrderUseCases = Depends(get_use_cases),
) -> OrderResponse:
    order = use_cases.get_order_by_id(order_id)
    if not order:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Order with id '{order_id}' not found.",
        )
    return order # type: ignore[return-value]

@router.post("/{order_id}/pay", response_model=OrderResponse)
def pay_order(
    order_id: str,
    use_cases: OrderUseCases = Depends(get_use_cases),
) -> OrderResponse:
    try:
        return use_cases.pay_order(order_id) # type: ignore[return-
value]
    except DomainError as err:
        if "not found" in str(err).lower():
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail=str(err)) from err
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail=str(err)) from err

@router.post("/{order_id}/cancel", response_model=OrderResponse)
def cancel_order(
    order_id: str,
    use_cases: OrderUseCases = Depends(get_use_cases),
) -> OrderResponse:
    try:
        return use_cases.cancel_order(order_id) # type: ignore[return-
value]
    except DomainError as err:
        if "not found" in str(err).lower():

```

```

        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail=str(err)) from err
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail=str(err)) from err

@router.get("", response_model=list[OrderResponse])
def list_orders(
    use_cases: OrderUseCases = Depends(get_use_cases),
) -> list[OrderResponse]:
    return use_cases.list_orders() # type: ignore[return-value]

```

Paso 5: Puntos de entrada de la aplicación (FastAPI Entrypoint)

Creamos el archivo src/orders_service/main.py.

```

from fastapi import FastAPI

from orders_service.infrastructure.api.routes import router as
orders_router

app = FastAPI(
    title="Orders Service API",
    description="Microservicio para la gestión de pedidos (Arquitectura
Hexagonal)",
    version="1.0.0",
)

app.include_router(orders_router)

@app.get("/health", tags=["Health"])
def health_check() -> dict[str, str]:
    return {"status": "ok"}

```

Paso 6: Pruebas de integración de Endpoints HTTP

Creamos el archivo tests/integration/api/test_routes.py

```

from collections.abc import Generator

import pytest
from fastapi.testclient import TestClient

```

```

from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker
from sqlalchemy.pool import StaticPool

from orders_service.infrastructure.database import Base, get_db_session
from orders_service.main import app

@pytest.fixture
def db_session() -> Generator[Session]:
    engine = create_engine(
        "sqlite:///memory:",
        connect_args={"check_same_thread": False},
        poolclass=StaticPool,
    )
    Base.metadata.create_all(engine)
    SessionLocal = sessionmaker(bind=engine)
    session = SessionLocal()
    try:
        yield session
    finally:
        session.close()

@pytest.fixture
def client(db_session: Session) -> Generator[TestClient]:
    def _override_get_db_session() -> Generator[Session]:
        yield db_session

    app.dependency_overrides[get_db_session] = _override_get_db_session
    with TestClient(app) as test_client:
        yield test_client
    app.dependency_overrides.clear()

def test_health_check(client: TestClient) -> None:
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}

def test_create_and_get_order_api(client: TestClient) -> None:

```

```

payload = {
    "customer_id": "cust-http-1",
    "items": [{"product_id": "prod-1", "quantity": 2, "price":
100.0}],
}
create_res = client.post("/orders", json=payload)
assert create_res.status_code == 201
data = create_res.json()
assert data["customer_id"] == "cust-http-1"
assert data["total_amount"] == "200.00"

order_id = data["id"]
get_res = client.get(f"/orders/{order_id}")
assert get_res.status_code == 200
assert get_res.json()["id"] == order_id

def test_pay_order_api(client: TestClient) -> None:
    payload = {
        "customer_id": "cust-http-2",
        "items": [{"product_id": "prod-1", "quantity": 1, "price":
50.0}],
    }
    create_res = client.post("/orders", json=payload)
    order_id = create_res.json()["id"]

    pay_res = client.post(f"/orders/{order_id}/pay")
    assert pay_res.status_code == 200
    assert pay_res.json()["status"] == "PAID"

```

Verificación local:

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run ruff check .

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run ruff check . --fix
Found 1 error (1 fixed, 0 remaining).

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run ruff check .
All checks passed!

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run mypy src
Success: no issues found in 14 source files

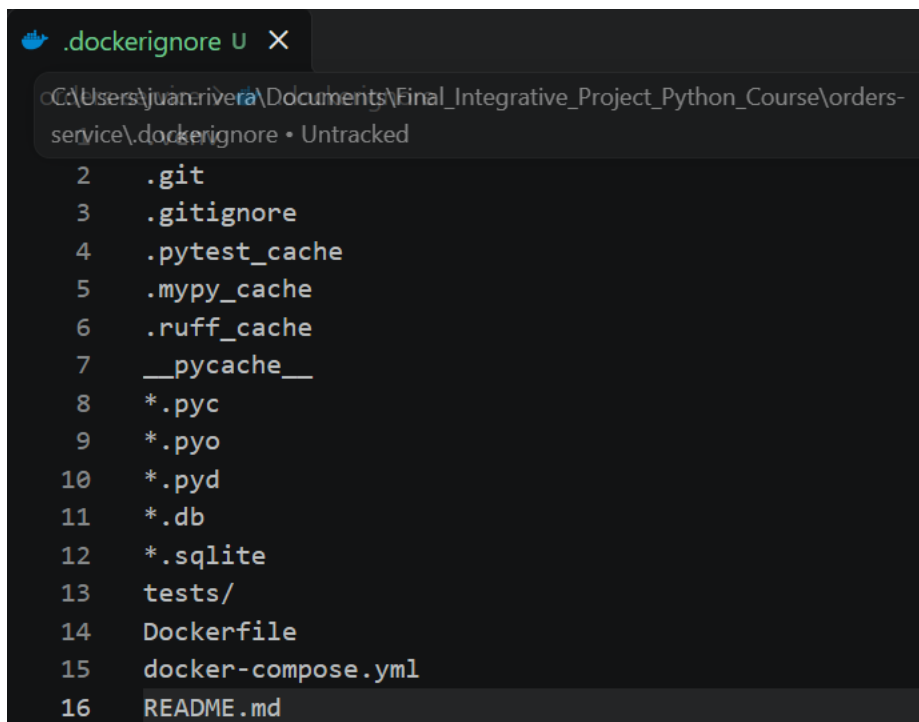
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ poetry run pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\juan.rivera\Documents\Final_Integrative_Project_Python_Course\orders-service
configfile: pyproject.toml
plugins: anyio-4.15.1, platformdirs-4.12.2
collected 15 items

tests\integration\api\test_routes.py ... [ 20%]
tests\integration\infrastructure\test_repository.py .. [ 33%]
tests\unit\application\test_use_cases.py ... [ 53%]
tests\unit\domain\test_order.py ..... [ 93%]
tests\unit\domain\test_ports.py . [100%]

===== warnings summary =====
```

Fase 6: Contenerización (Docker y Docker Compose)

Paso 1: Crear el archivo .dockerignore



```
.dockerignore
2  .git
3  .gitignore
4  .pytest_cache
5  .mypy_cache
6  .ruff_cache
7  __pycache__
8  *.pyc
9  *.pyo
10 *.pyd
11 *.db
12 *.sqlite
13 tests/
14 Dockerfile
15 docker-compose.yml
16 README.md
```

Paso 2: Crear el Dockerfile

Creamos el archivo Dockerfile en la raíz del proyecto. Utilizamos un multi-stage build con python: 3.13 para la imagen final sea liviana, rápida y segura (ejecutándose con un usuario sin privilegios de root):

```
# Stage 1: Builder
FROM python:3.13-slim AS builder

WORKDIR /app

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    POETRY_VERSION=2.0.0 \
    POETRY_HOME="/opt/poetry" \
    POETRY_NO_INTERACTION=1 \
    POETRY_VIRTUALENVS_IN_PROJECT=true

RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*

RUN curl -sSL https://install.python-poetry.org | python3 -
ENV PATH="$POETRY_HOME/bin:$PATH"

COPY pyproject.toml poetry.lock ./
RUN poetry install --only main --no-root

# Stage 2: Runner
FROM python:3.13-slim AS runner

WORKDIR /app

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PATH="/app/.venv/bin:$PATH"

RUN groupadd -g 1000 appuser && \
    useradd -u 1000 -g appuser -s /bin/sh appuser

COPY --from=builder /app/.venv /app/.venv
COPY alembic.ini ./
COPY alembic/ ./alembic
COPY src/ ./src

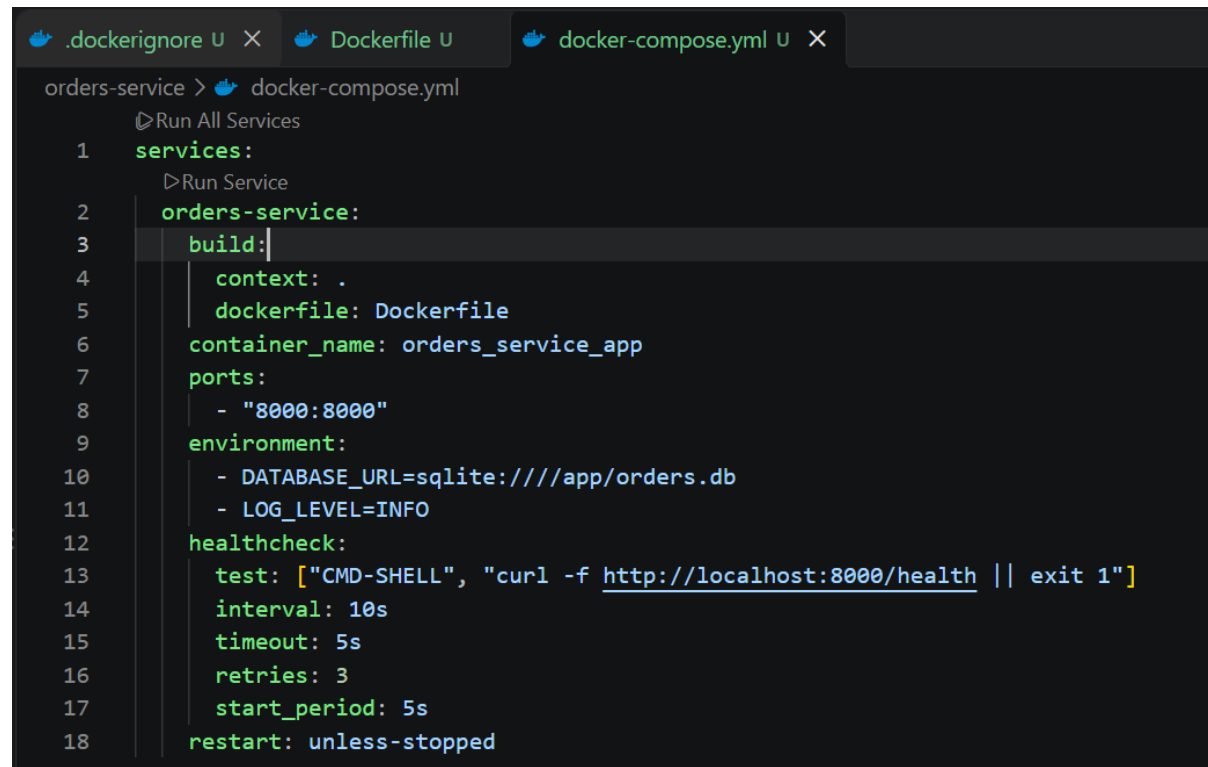
RUN chown -R appuser:appuser /app
```

USER appuser

EXPOSE 8000

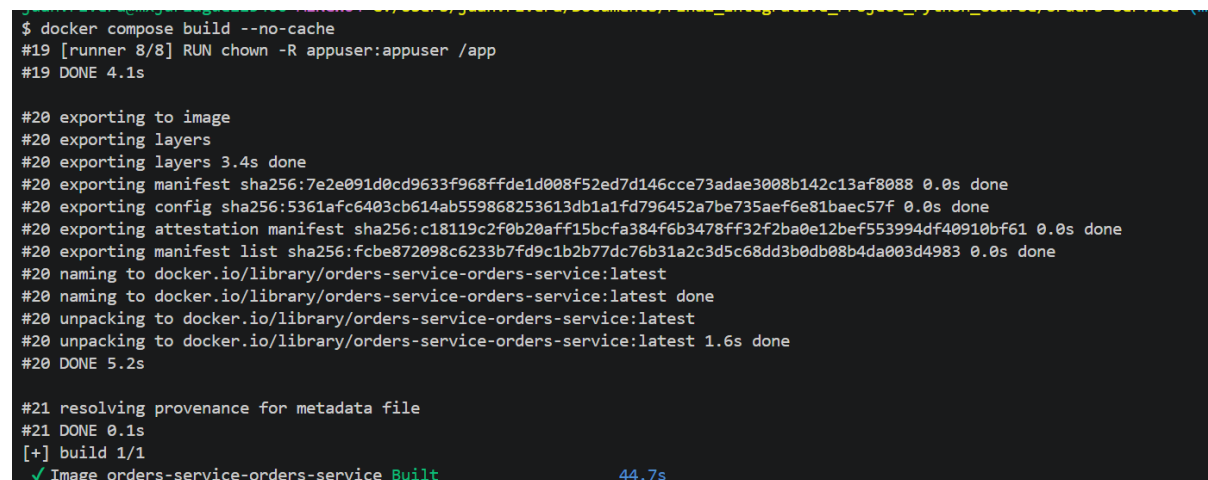
CMD ["uvicorn", "orders_service.main:app", "--host", "0.0.0.0", "--port", "8000"]

Paso 3: Crear el docker-compose.yml



```
orders-service > docker-compose.yml
  Run All Services
1  services:
  Run Service
2    orders-service:
3      build:
4        context: .
5        dockerfile: Dockerfile
6      container_name: orders_service_app
7      ports:
8        - "8000:8000"
9      environment:
10       - DATABASE_URL=sqlite:///app/orders.db
11       - LOG_LEVEL=INFO
12      healthcheck:
13        test: ["CMD-SHELL", "curl -f http://localhost:8000/health || exit 1"]
14        interval: 10s
15        timeout: 5s
16        retries: 3
17        start_period: 5s
18      restart: unless-stopped
```

Paso 4: Validar localmente con Docker



```
$ docker compose build --no-cache
#19 [runner 8/8] RUN chown -R appuser:appuser /app
#19 DONE 4.1s

#20 exporting to image
#20 exporting layers
#20 exporting layers 3.4s done
#20 exporting manifest sha256:7e2e091d0cd9633f968ffde1d008f52ed7d146cce73adae3008b142c13af8088 0.0s done
#20 exporting config sha256:5361afc6403cb614ab559868253613db1afd796452a7be735aef6e81baec57f 0.0s done
#20 exporting attestation manifest sha256:c18119c2f0b20aff15bcfa384f6b3478ff32f2ba0e12bef553994df40910bf61 0.0s done
#20 exporting manifest list sha256:fcbe872098c6233b7fd9c1b2b77dc76b31a2c3d5c68dd3b0db08b4da003d4983 0.0s done
#20 naming to docker.io/library/orders-service-orders-service:latest
#20 naming to docker.io/library/orders-service-orders-service:latest done
#20 unpacking to docker.io/library/orders-service-orders-service:latest
#20 unpacking to docker.io/library/orders-service-orders-service:latest 1.6s done
#20 DONE 5.2s

#21 resolving provenance for metadata file
#21 DONE 0.1s
[+] build 1/1
✓ Image orders-service-orders-service Built 44.7s
```

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ docker compose up -d
[+] up 1/1
✓ Container orders_service_app Started 0.2s
```

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS               PORTS
orders_service_app  orders-service-orders-service      "uvicorn orders_serv..."  orders-service  16 seconds ago  Up 15 seconds (healthy)  0.0.0.0:8000->8000/tcp

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Final_Integrative_Project_Python_Course/orders-service (main)
$ curl http://localhost:8000/health
{"status":"ok"}
```